

Assignment II

Problem Bank 20

- **Assignment Description:**

The assignment aims to provide deeper understanding of Pipelining Architecture, Scheduling, Multithreading and deadlock condition using CPU- OS Simulator. The assignment has three parts.

- Part I deals with Pipeline Architecture
- Part II deals with Scheduling algorithm (FCFS, SJF and RR)
- Part III deals with Multithreading
- Part IV deals with Deadlock condition

- **Submission:** You will have to submit this documentation file and the name of the file should be GROUP-NUMBER.pdf. For Example, if your group number is 1, then the file name should be GROUP-1.pdf.

Submit the assignment by **August 24, 2025 through Taxila only**. File submitted by any means outside Taxila will not be accepted and marked. In case of any issues, please drop an email to the course lead LF Vaibhav Jain (email: vaibhav.jain@wilp.bits-pilani.ac.in)

Caution!!!

Assignments are designed for individual groups which may look similar, and you may not notice minor changes in the assignments. Hence, refrain from copying or sharing documents with others. Any evidence of such practice will attract severe penalty. Remember that any kind of group changes after the announcement of the assignment is not allowed.

Evaluation:

- The assignment carries 15 marks
- Grading will depend on
 - Contribution of each student in the implementation of the assignment
 - **Plagiarism or copying will result in -15 marks**

*****FILL IN THE DETAILS GIVEN BELOW*****

Group Name: Group 120

Contribution Table:

Contribution (This table should contain the list of all the students in the group. Clearly mention each student's contribution towards the assignment. Mention "No Contribution" in cases applicable. If the contribution is equal the write 100%)

Sl. No.	Name (as appears in Canvas)	ID NO	Contribution (%)
1.	RAJESHWARI M	2024DC04277	100%
2.	KANHAIYA SHARMA	2024DC04237	100%
3.	AKASH MALIK	2024DC04030	100%

Resource for Part I, II, III & IV:

- Use following link to login to "eLearn" portal.
 - <https://elearn.bits-pilani.ac.in>
- Under tab 'My Courses', select Labware and then go to the course page "**Labware_Computer Organization and Software Systems**". The direct link to access the course is <https://taxila-aws.bits-pilani.ac.in/course/view.php?id=12598>
- Click on "[CSIS-Connect to Virtual Lab\(OSHA\)](#)" to login to lab portal using elearn credentials.
- Use Cloud virtual lab access manual to use the lab platform and access the Cloud virtual machine.
- In cloud lab machine, click on "Lab Resources" -> Click on "Computer Organization and software systems" course folder.

Part I: Pipeline Processor

Consider the following program:

```
program pipeline20
  x=5
  y=2
  z=10
  w=5
  z=z + 1
  y=y - 1
  z=z + x
  w=x + y
  z= z + 1
end
```

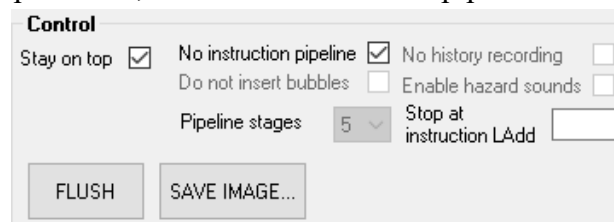
Compile the code and load it in CPU-OS simulator. Perform the following:

Execute the above program using non-pipelined processor and pipelined processor and answer the following questions.

Note: Every time flush the pipeline before running the code

A) Non-pipelined Processor:

To enable non-pipelined processor, check “No instruction pipeline” check box in control panel.



- a) How many stages are there in non-pipelined processor? List them. **[0.5 Mark]**

Solution: **5 => Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), Write Back (WB)**

- b) Fill in the following after executing of above program using non-pipelined processor. **[01 Mark]**

	Clocks	Instruction Count	CPI	Speed up Factor	Inst. Syncs
Non-Pipelined Processor	161	30	5.55	0.9	1

- c) What are the contents of General-purpose registers after the execution of the program?

[01 Mark]

Solution:

R01: 5, R02: 1, R03: 17, R04: 6, R05: 17, R06: 17, R07: 0

B) Pipelined processor:

To use, enable pipelined processor, uncheck “No instruction pipeline” check box in control panel.

a) Fill in the following table with respect to pipelined processor execution of the above program:

[0.5 + 0.5 Mark]

Pipelined processor conditions	Clocks	Instruction Count	CPI	Speed up Factor	Inst. Syncs	Data hazard (Yes/No)	Contents of registers used by the program
Check “Do not insert bubbles” check box	5	1	5	1	1	0	R04= 6, R05= 17, R06= 17
Uncheck “Do not insert bubbles”	5	1	5	1	1	0	R04= 6, R05= 17, R06= 17

b) Write your observation on the impact of bubble insertion. Is there a way to improve the CPI and Speed up factor? If so give the solution. [01 Mark]

Solution:

Impact on bubble insertion

- Bubbles are inserted to resolve data hazards, control hazards, or structural hazards.
- They stall the pipeline, increasing the execution time and CPI, thereby reducing speed-up.

To Improve CPI and Speed Up we can some methods like Instruction Reordering (to avoid dependencies), dynamic branch prediction (to reduce control hazards), data forwarding (pass results directly between pipeline stages without waiting for write-back).

Part II: Process Scheduling

Consider the following source codes:

```
program LoopTest  
  i = 5  
  for n = 1 to 12  
    x = i + n  
    i = i + 2  
    n = n + 2  
  next  
end
```

Compile the above source code and load it in the main memory.

We are now going to use the OS simulator to run this code. To enter the OS simulator:

- 1) Click on the OS O... button in the current window. The OS window opens.
- 2) You should see an entry, titled LoopTest, in the PROGRAM LIST view.
- 3) Now that this program is available to the OS simulator, we can create as many instances, i.e. processes, of it as we like. You do this by clicking on the CREATE NEW PROCESS button.

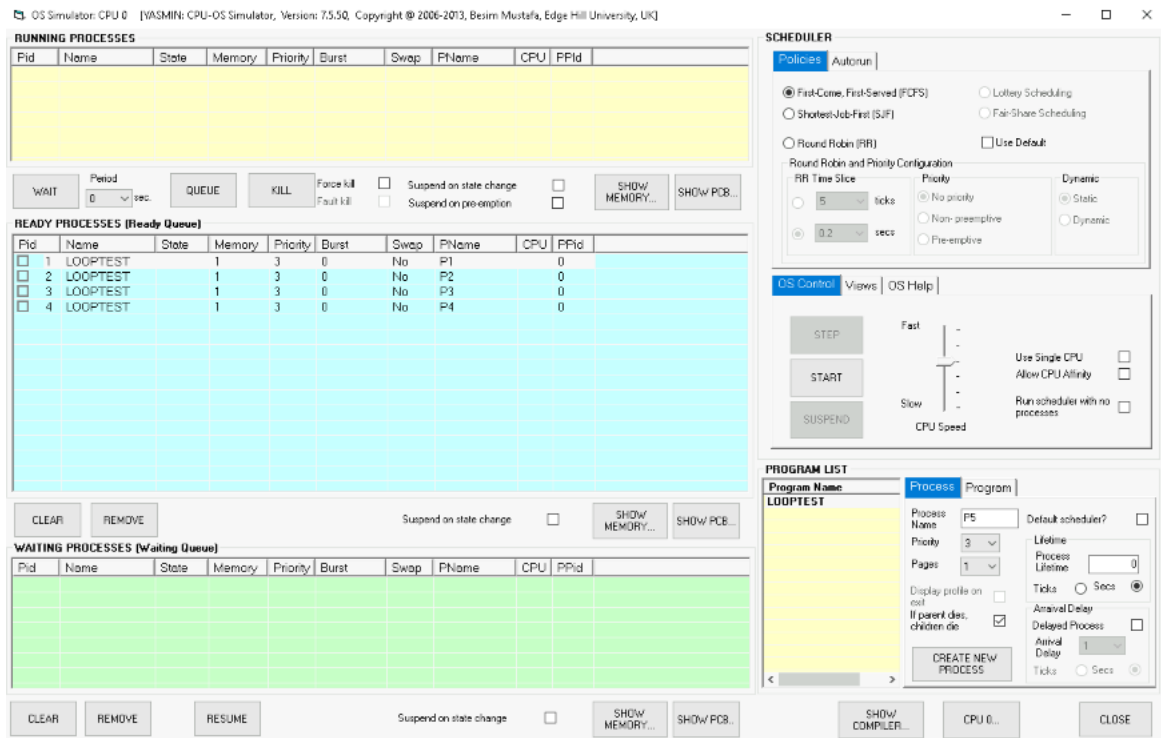
PART-II_A

Answer the following with reference to FCFS scheduling algorithm:

Initialize the OS simulator with the following:

- Select the **First-Come-First-Served (FCFS)** option in the SCHEDULER/Policies view.
- Time slice should be considered as **seconds**.
- Create four processes P1, P2, P3 and P4 from source code respectively. Set the arrival delay as zero seconds.
- **Arrival delay** should be considered in **seconds** in the OS simulator.

Present the snapshot of the above settings.



- a) Slide the Speed selector half-way down and then hit the START button. What is the order in which processes are executed? [0.5 Mark]

P1→P2→P3→P4

- b) What is the *Elapsed time*, *Average Process Waiting Time* and *Average Burst Period* and of each process? (To see this, Click on VIEWS button available on the left of your OS control, the click VIEW LOG) [0.5 Mark]

Process	<i>Arrival Time/Delay</i>	<i>Elapsed Time (sec)</i>	<i>Average Waiting Time (sec)</i>	<i>Average Burst Period</i>
P1	0	78	0.28	71
P2	0	77	79.08	71
P3	0	77	156.88	71
P4	0	77	234.28	71

PART-II_B

Answer the following with reference to SJF scheduling algorithm:

Initialize the OS simulator with the following:

- Select the **Shortest Job First (SJF)** option in the SCHEDULER/Policies view.
- Select the Priority (static) as **Non-Preemptive** option in the SCHEDULER/Policies view.
- Time slice should be considered as **seconds**.

- Create four processes P1, P2, P3 and P4 from source codes respectively. Set the arrival delay as zero seconds.
- **Arrival delay** should be considered in **seconds** in the OS simulator.

(a) Slide the Speed selector half-way down and then hit the START button. What is the order in which processes are executed? **[0.5 Mark]**

P1→P2→P3→P4

(b) What is the *Elapsed time*, *Average Process Waiting Time* and *Average Burst Period* and of each process? (To see this, Click on VIEWS button available on the left of your OS control, the click VIEW LOG) **[0.5 Mark]**

Process	<i>Arrival Time/Delay</i>	<i>Elapsed Time (sec)</i>	<i>Average Waiting Time (sec)</i>	<i>Average Burst Period</i>
P1	0	78	0.29	71
P2	0	77	78.29	71
P3	0	77	155.49	71
P4	0	77	232.69	71

PART-II_C

Answer the following with reference to RR scheduling algorithm:

Initialize the OS simulator with the following:

- Select the **Round Robin (RR)** with **10 seconds** as time slice option in the SCHEDULER/Policies view.
- Select the Priority (static) as **Non-Preemptive** option in the SCHEDULER/Policies view.
- Time slice should be considered as **seconds**.
- Create four processes P1, P2, P3 and P4 from source codes respectively. Set the arrival delay as zero seconds.
- **Arrival delay** should be considered in **seconds** in the OS simulator.

(a) Slide the Speed selector half-way down and then hit the START button. What is the order in which processes are executed? **[0.5 Mark]**

P1→P2→P3→P4→P1→P2→P1→P2→ P1→P2→ P1→P2→ P1→P2→
P1→P2→P1→P2→P2→P3→ P3→ P3→ P3→ P3→ P3→ P3→ P3→P4

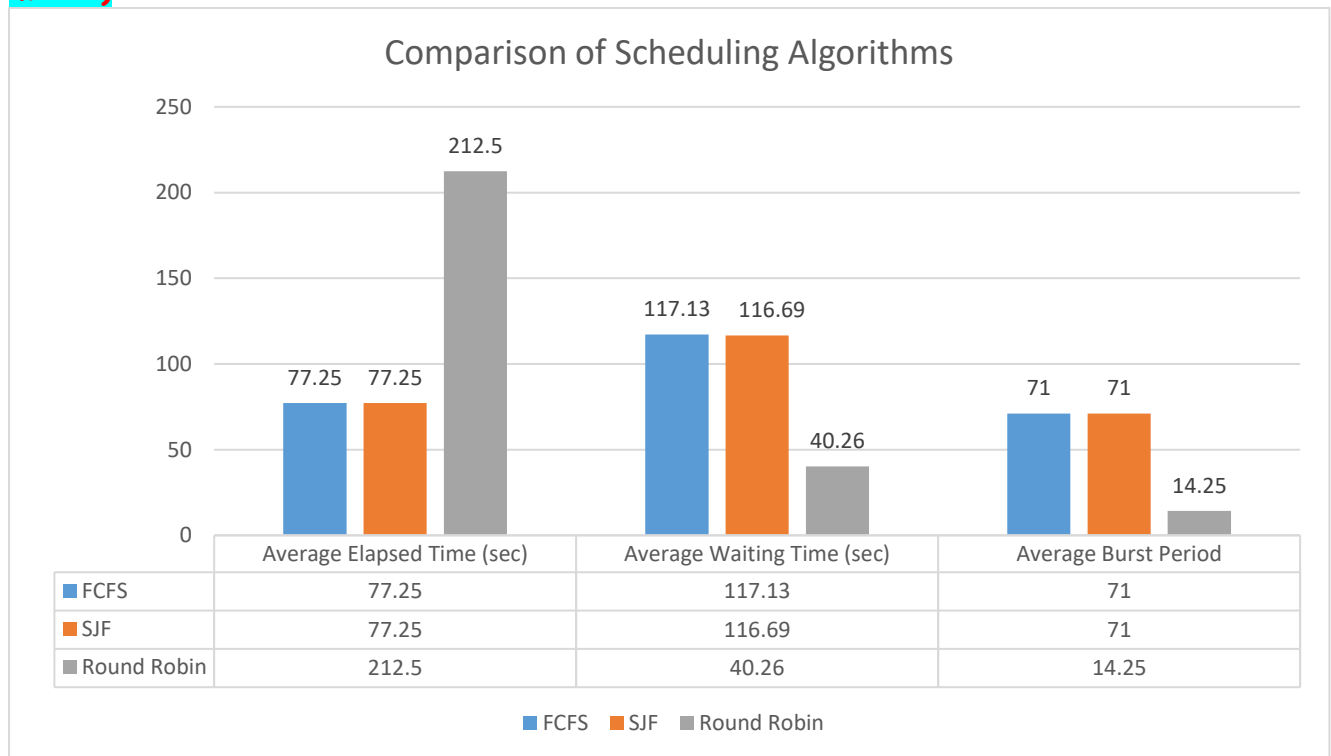
(b) What is the *Elapsed time*, *Average Process Waiting Time* and *Average Burst Period* and of each process? (To see this, Click on VIEWS button available on the left of your OS control, the click VIEW LOG) **[0.5 Mark]**

Process	<i>Arrival Time/Delay</i>	<i>Elapsed Time (sec)</i>	<i>Average Waiting Time (sec)</i>	<i>Average Burst Period</i>
P1	0	170	11.41	8
P2	0	169	11.45	7
P3	0	228	19.03	7
P4	0	283	119.14	35

PART-II_D

- a) Plot a graph from the results obtained by FCFS, SJF and Round Robin scheduling and explain which algorithm is better among these with proper justification. [01 Mark]

(Use any platform to obtain single graph, multiple graphs will fetch 0 marks)



Conclusion :

For Interactive system Round Robin is preferable. Because it cuts **average waiting time** to **40.26 s** versus **~117 s** for FCFS/SJF, so processes get CPU sooner and the system feels more responsive.

If minimizing turnaround time is the only goal. SJF is the better choice in principle because it **minimizes average turnaround time** (and waiting time) by running the shortest jobs first.

Part III: Multi-Threading

Consider the following source code

```
program ThreadTest
    total = 0
    sub thread1 as thread
        for i = 1 to 5
            total = total + i
        next
    end sub
    sub thread2 as thread
        for i = 5 to 10
            total = total - i
            call thread1
        next
    end sub
    sub thread3 as thread
        call thread2
        for i = 10 to 15
            total = total + i
        next
    end sub
    sub thread4 as thread
        call thread3
        for i = 15 to 20
            total = total - i
        next
    end sub

    call thread4

    wait
    writeln ("Total =", total)
end
```

Compile the above source code and load it in the main memory. Create a single process, choose **RR** scheduling algorithm with time quantum of **05 ticks with no-priority**. Run the Process.

Answer the following questions: [01 Mark Each]

a) What is the value of “Total” ?

-60

b) How many processes and how many threads are created?

1 process and 7 threads

c) Identify the name of the processes and threads.

Process: P1

Threads: P1T0, P1T0T1, P1T0T1T2, P1T0T1T2T3, P1T0T1T2T4, P1T0T1T2T5, P1T0T1T2T6.

d) What is the PID and PPID of the processes and threads created?

- **P1:** PID = 2, PPID = 1
- **P1T0:** PID = 3, PPID = 2
- **P1T0T1:** PID = 4, PPID = 3
- **P1T0T1T2:** PID = 5, PPID = 4
- **P1T0T1T2T3:** PID = 6, PPID = 5
- **P1T0T1T2T4:** PID = 7, PPID = 5
- **P1T0T1T2T5:** PID = 8, PPID = 5
- **P1T0T1T2T6:** PID = 9, PPID = 5

e) Represent the parent and child relationship using tree representation?

P1(2)

└─ P1T0(3)

└─ P1T0T1(4)

└─ P1T0T1T2(5)

└─ P1T0T1T2T3(6)

└─ P1T0T1T2T4(7)

└─ P1T0T1T2T5(8)

└─ P1T0T1T2T6(9)

PART-IV: DeadLock

Consider the following 03 programs, Replace **<P1_GroupName>** by “P1_ followed by your group number”. For example, if group number is Group99 then P1_Group99. Similarly replace **<P2_GroupName>** and **<P3_GroupName>**. Programs names without your group number will fetch zero marks

```
program P1_Group120
    resource(1, allocate)
    wait(3)
    resource(2, allocate)
    for n = 1 to 20
    next
end
```

```
program P2_Group120
    resource(2, allocate)
    wait(3)
    resource(3, allocate)
    for n = 1 to 20
    next
end
```

```
program P3_Group120
    resource(3, allocate)
    wait(3)
    resource(4, allocate)
    for n = 1 to 20
    next
end
```

Perform / Answer the following:

Task 1: Draw the resource allocation graph for the above code. **[0.5 Marks]**

R1 -->|Allocated| P1

P1 -->|Requests| R2

R2 -->|Allocated| P2

P2 -->|Requests| R3

R3 -->|Allocated| P3

P3 -->|Requests| R4

Task 2: Compile all the above three source codes and load in the main memory at starting addresses 200, 300, and 400 respectively. Provide the snapshot of PROGRAM SOURCE [INPUT] and PROGRAM CODE [OUTPUT] for all the three source codes. [paste entire compiler simulator window, after compiling each code]. **[0.25 Marks]**

Program Compiler [RISCV] CPU-03 Simulator, Version: 7.3.38, Copyright © 2006-2015, Boris Murfin, Edge Hill University, UK

PROGRAM SOURCE (INPUT)

```
program P1_Group120
resource(1, allocate)
wait(1)
resource(2, allocate)
for n = 1 to 20
next
end
```

COMPILER PROGRESS

1....Scanning for keyword 'for' - Found 1
1....Found keyword 'next' 10
1....Found keyword 'end' 20
Code generation completed...
Displaying generated code
Display completed...

*** NOTE: Click on numbers in brackets to highlight corresponding source lines in source editor.

COMPILE [Source] [Options] [Options] [Compiler Help]

COMPILE RUN SHOW SYMBOL TABLE... SHOW CLAS... MAP... SHOW SOURCE FILE LIST... SHOW ATT...

SOURCE

LOAD... SAVE... PRINT...

ASSEMBLY CODE

Start Address: 0 Code Size: 72 PRINT Include source Select CPU: 0 From Address: 200 LOAD IN MEMORY... SHOW CPU... SHOW CPU...

BINARY CODE

LOAD... SHOW... SAVE... Use compiler optimisations... CLOSE

PROGRAM CODE (OUTPUT)

Label	CPU Instruction	Binary Code	Line	Comments
**** CODE:				
**** MAIN PROG...				
0000	PSH #1	0F000001	0002	Push the value of 1 on top of stack
0004	SVI 12	2102000C	0002	OS system call: Resource allocate w...
0008	POP R01	0F0101	0002	Pop the value from top of stack into ...
0011	MOV #3, R04	0000000304	0003	Copy the value of 3 to _STempReg...
0017	PSH R04	0E0304	0003	Wait time
0020	SVI 6	21030006	0003	OS system call: Wait
0024	PSH #2	0F000002	0004	Push the value of 2 on top of stack
0028	SVI 12	2102000C	0004	OS system call: Resource allocate w...
0033	POP R02	0F0102	0004	Pop the value from top of stack into ...
0035	MOV #1, R06	0000000106	0005	Copy the value of 1 to _STempReg...
0041	MOV R06, R04	00010604	0005	Copy the value of _STempReg245 t...
0046	MOV #20, R06	0000001A06	0005	Copy the value of 20 to _STempRe...
0053	CMP R06, R04	30010604	0005	Compare N with _STempReg245
0057	JGT #1	20020047	0005	Jump to code at address 71 if status ...
0061	ADD #1, R04	1100000104	0006	Add N = N + 1
0067	JMP #2	17020034	0006	Jump to code at address 52 (ENCO...
0071	HLT	2F	0007	Stop simulation
**** DATA:				

Program Compiler [RISCV] CPU-03 Simulator, Version: 7.3.38, Copyright © 2006-2015, Boris Murfin, Edge Hill University, UK

PROGRAM SOURCE (INPUT)

```
program P2_Group120
resource(2, allocate)
wait(1)
resource(3, allocate)
for n = 1 to 20
next
end
```

COMPILER PROGRESS

1....Scanning for keyword 'for' - Found 1
1....Scanning for keyword 'next' - Found 10
1....Found keyword 'end' 20
1....Scanning for keyword 'for' - Found 1
1....Scanning for keyword 'next' - Found 10
1....Scanning for keyword 'for' - Found 1
1....Scanning for keyword 'next' - Found 10
1....Found FOR statement 10
1....Found keyword 'end' 20
1....Found Assignment statement 10
1....Scanning for keyword 'for' - Found 1
1....Found keyword 'next' 10
1....Found keyword 'end' 20
Code generation completed...
Displaying generated code
Display completed...

*** NOTE: Click on numbers in brackets to highlight corresponding source lines in source editor.

COMPILE [Source] [Options] [Options] [Compiler Help]

COMPILE RUN SHOW SYMBOL TABLE... SHOW CLAS... MAP... SHOW SOURCE FILE LIST... SHOW ATT...

SOURCE

LOAD... SAVE... PRINT...

ASSEMBLY CODE

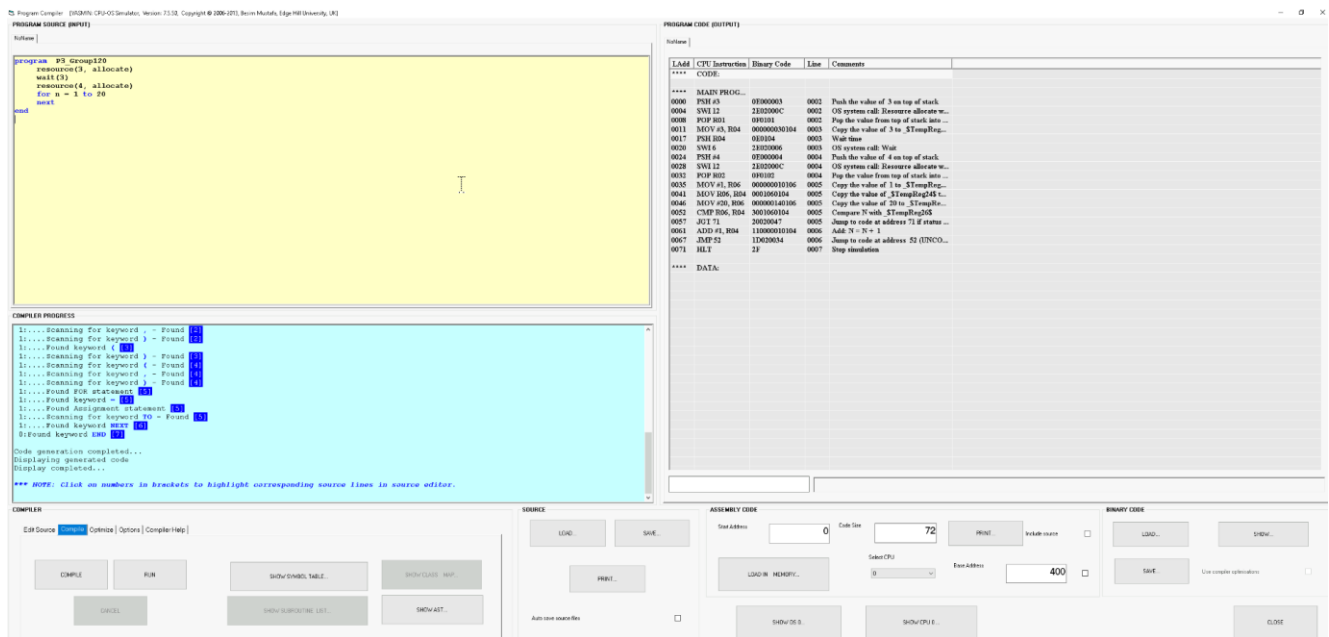
Start Address: 0 Code Size: 72 PRINT Include source Select CPU: 0 From Address: 300 LOAD IN MEMORY... SHOW CPU... SHOW CPU...

BINARY CODE

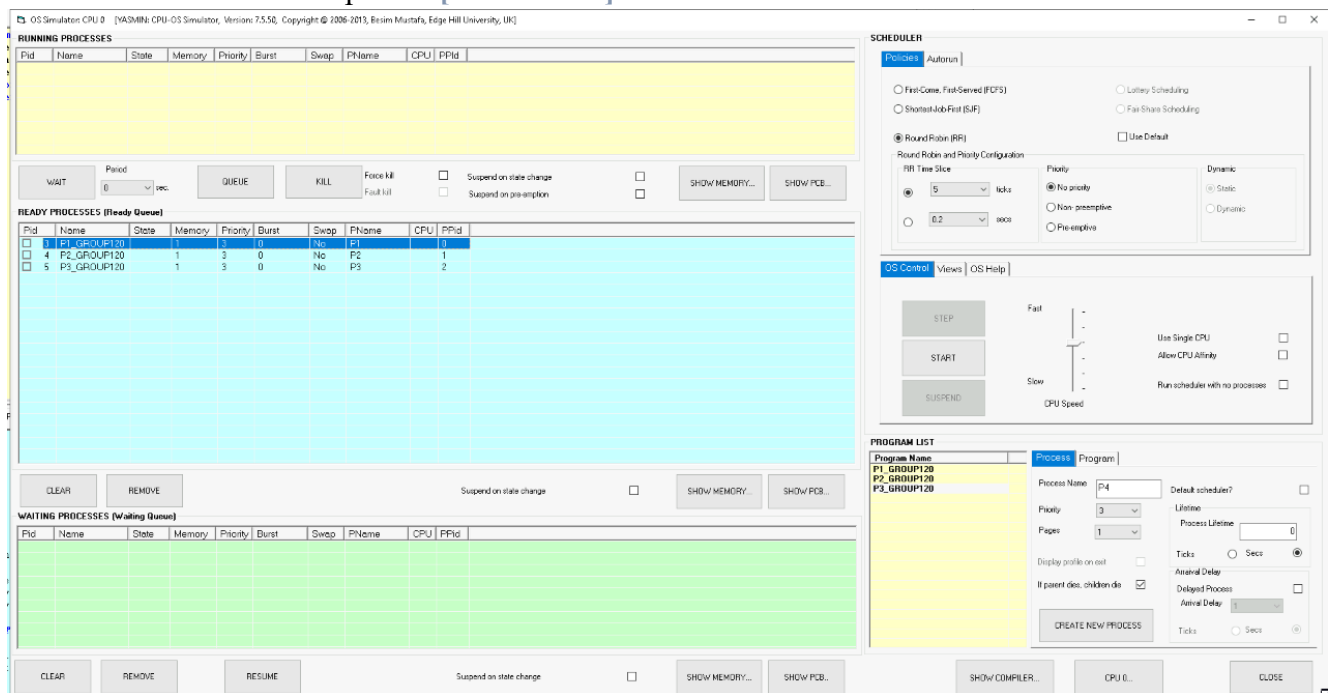
LOAD... SHOW... SAVE... Use compiler optimisations... CLOSE

PROGRAM CODE (OUTPUT)

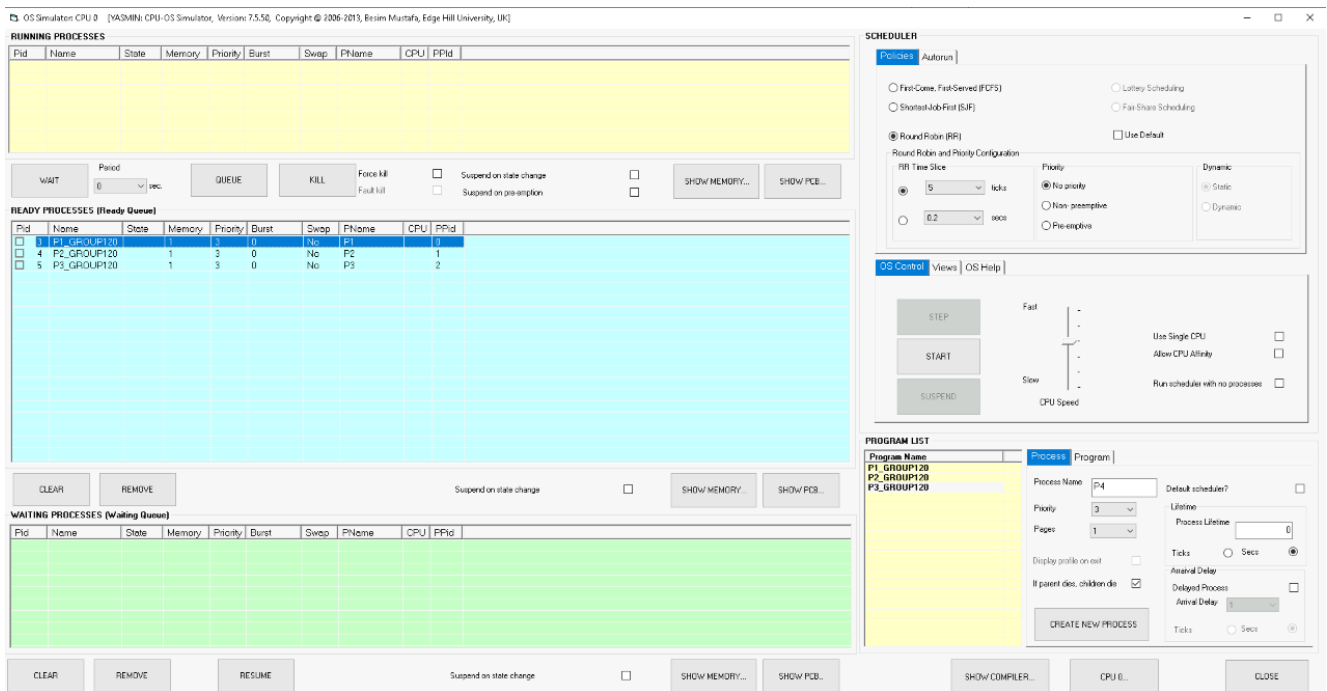
Label	CPU Instruction	Binary Code	Line	Comments
**** CODE:				
**** MAIN PROG...				
0000	PSH #2	0F000002	0002	Push the value of 2 on top of stack
0004	SVI 12	2102000C	0002	OS system call: Resource allocate w...
0008	POP R01	0F0101	0002	Pop the value from top of stack into ...
0011	MOV #3, R04	0000000304	0003	Copy the value of 3 to _STempReg...
0017	PSH R04	0E0304	0003	Wait time
0020	SVI 6	21030006	0003	OS system call: Wait
0024	PSH #3	0F000003	0004	Push the value of 3 on top of stack
0028	SVI 12	2102000C	0004	OS system call: Resource allocate w...
0033	POP R03	0F0103	0004	Pop the value from top of stack into ...
0035	MOV #1, R06	0000000106	0005	Copy the value of 1 to _STempReg...
0041	MOV R06, R04	00010604	0005	Copy the value of _STempReg245 t...
0046	MOV #20, R06	0000001A06	0005	Copy the value of 20 to _STempRe...
0053	CMP R06, R04	30010604	0005	Compare N with _STempReg245
0057	JGT #1	20020047	0005	Jump to code at address 71 if status ...
0061	ADD #1, R04	1100000104	0006	Add N = N + 1
0067	JMP #2	17020034	0006	Jump to code at address 52 (ENCO...
0071	HLT	2F	0007	Stop simulation
**** DATA:				



Task 3: Create 3 processes P1, P2, and P3 from the respective programs. Provide the evidence of process creation in the form of snapshot. [0.25 Marks]



Task 4: Select round robin scheduling algorithm and set time slice to 05 ticks with No-Priority. Provide the evidence of setting in the form of snapshot. [0.25 Marks]



Task 5: Start the process execution. Open the system resources window by pressing VIEW RESOURCES... button under views tab. Then press SHOW DEADLOCKED PROCESSES button. Observe the resource allocation graph. Present the snapshot of System resources window and resource allocation graph at the end of the execution of processes. [0.25 Marks]

